

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Dataset Integration

**COURSE
NAME:**

Query Processing for Data
Science

**COURSE
CODE:**

DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 30%

Total Marks: 20

Code of Conduct

I S. Sharon Surya (192324246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Sharon Surya

Dataset Integration Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Dataset Understanding & Preparation	30%	Correctly understands both datasets, identifies relevant columns/keys, and loads them properly.	Minor misunderstanding or preparation issue.	Loads datasets but needs some guidance in identifying relevant fields.	Unable to identify or prepare the datasets correctly.
Dataset Integration	20%	Correctly integrates datasets using appropriate merge(), join(), or concat() and selects the correct integration key.	Integration is mostly correct with minor errors.	Performs integration with significant guidance or minor key/operation errors.	Unable to integrate the datasets correctly.
Data Processing & Analysis	15%	Correctly performs all required calculations, filtering,	Performs most required analysis correctly.	Performs basic analysis but misses some	Analysis is incorrect or incomplete.

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
		grouping, or transformations on the integrated data.		required operations.	
Python/Pandas Implementation	20%	Code is correct, efficient, well-structured, and uses appropriate Pandas functions.	Code works with minor inefficiencies/errors.	Code works partially and requires corrections.	Code contains major errors or does not execute.
Output & Interpretation	15%	Clearly presents the required output and correctly interprets the results.	Output is correct with limited interpretation.	Output is partially correct or poorly presented.	Output is missing/incorrect and results are not interpreted.

Total Marks Awarded:

Signature of the Course Faculty

Questions:

Set 1 – Dataset Integration Tasks

1. Employee–Department Integration

You are given two datasets:

- employees.csv — Employee_ID, Employee_Name, Department_ID, Salary
- departments.csv — Department_ID, Department_Name, Location

Task: Write a Python program using Pandas to integrate the two datasets based on Department_ID and create a new dataset containing employee name, department name, location, and salary.

CODE:

```
import pandas as pd

# Read the datasets
employees = pd.read_csv("employees.csv")
departments = pd.read_csv("departments.csv")

# Merge the two datasets using Department_ID
integrated_data = pd.merge(
    employees,
    departments,
    on="Department_ID",
    how="inner"
)

# Select the required columns
result = integrated_data[
    ["Employee_Name", "Department_Name", "Location", "Salary"]
]

# Display the integrated dataset
print(result)

# Save the result to a new CSV file
result.to_csv("employee_department_integrated.csv", index=False)

print("\nIntegrated dataset saved successfully!")
```

OUTPUT:

```

# Merge the two datasets using Department_ID
integrated_data = pd.merge(
    employees,
    departments,
    on="Department_ID",
    how="inner"
)

# Select the required columns
result = integrated_data[
    ["Employee_Name", "Department_Name", "Location", "Salary"]
]

# Display the integrated dataset
print(result)

# Save the result to a new CSV file
result.to_csv("employee_department_integrated.csv", index=False)

print("\nIntegrated dataset saved successfully!")

```

	Employee_Name	Department_Name	Location	Salary
0	Ravi	IT	Chennai	50000
1	Priya	HR	Bangalore	55000
2	Arun	IT	Chennai	48000
3	Sneha	Finance	Hyderabad	60000
4	Karthik	HR	Bangalore	52000

Integrated dataset saved successfully!

2. Customer–Transaction Integration

You are given:

customers.csv — Customer_ID, Customer_Name, City

transactions.csv — Transaction_ID, Customer_ID, Product, Amount

Task: Integrate the datasets using Customer_ID. Display the customer name, city, product, and transaction amount. Identify customers who have made transactions above ₹10,000.

CODE:

```

import pandas as pd

# Read the datasets
customers = pd.read_csv("customers.csv")
transactions = pd.read_csv("transactions.csv")

# Integrate the datasets using Customer_ID
integrated_data = pd.merge(
    customers,
    transactions,
    on="Customer_ID",
    how="inner"
)

# Display required columns
result = integrated_data[
    ["Customer_Name", "City", "Product", "Amount"]
]

```

```

print("Integrated Customer-Transaction Data:")
print(result)

# Identify transactions above ₹10,000
high_value = result[result["Amount"] > 10000]

print("\nCustomers with transactions above ₹10,000:")
print(high_value)

# Save the integrated dataset
result.to_csv("customer_transaction_integrated.csv", index=False)

# Save high-value transactions
high_value.to_csv("transactions_above_10000.csv", index=False)

print("\nFiles saved successfully!")

```

OUTPUT:

```

high_value = result[result["Amount"] > 10000]

print("\nCustomers with transactions above ₹10,000:")
print(high_value)

# Save the integrated dataset
result.to_csv("customer_transaction_integrated.csv", index=False)

# Save high-value transactions
high_value.to_csv("transactions_above_10000.csv", index=False)

print("\nFiles saved successfully!")

```

```

... Integrated Customer-Transaction Data:
  Customer_Name  City  Product  Amount
0         Ravi  Chennai  Laptop   55000
1         Ravi  Chennai  Printer   11000
2        Priya  Bangalore  Mobile   15000
3         Arun  Hyderabad  Headphones   5000
4        Sneha  Chennai  Tablet   12000
5       Karthik  Mumbai  Monitor    8500

```

```

Customers with transactions above ₹10,000:
  Customer_Name  City  Product  Amount
0         Ravi  Chennai  Laptop   55000
1         Ravi  Chennai  Printer   11000
2        Priya  Bangalore  Mobile   15000
4        Sneha  Chennai  Tablet   12000

```

```
Files saved successfully!
```

3. Student–Marks Integration

You are given:

students.csv — Student_ID, Student_Name, Department

marks.csv — Student_ID, Python, DBMS, Statistics

Task: Merge the datasets using Student_ID. Create a new column Total_Marks and identify students whose total marks are greater than 240.

CODE:

```
import pandas as pd

# Read the datasets
students = pd.read_csv("students.csv")
marks = pd.read_csv("marks.csv")

# Merge the datasets using Student_ID
integrated_data = pd.merge(
    students,
    marks,
    on="Student_ID",
    how="inner"
)

# Calculate Total Marks
integrated_data["Total_Marks"] = (
    integrated_data["Python"] +
    integrated_data["DBMS"] +
    integrated_data["Statistics"]
)

# Display the integrated dataset
print("Integrated Student-Marks Data:")
print(integrated_data)

# Identify students with Total Marks greater than 240
high_scorers = integrated_data[
    integrated_data["Total_Marks"] > 240
]

print("\nStudents with Total Marks greater than 240:")
print(high_scorers)

# Save the complete integrated dataset
integrated_data.to_csv("student_marks_integrated.csv", index=False)

# Save students scoring above 240
high_scorers.to_csv("students_above_240.csv", index=False)

print("\nFiles saved successfully!")
```

OUTPUT:

```
high_scorers = integrated_data[
    integrated_data["Total_Marks"] > 240
]

print("\nStudents with Total Marks greater than 240:")
print(high_scorers)

# Save the complete integrated dataset
integrated_data.to_csv("student_marks_integrated.csv", index=False)

# Save students scoring above 240
high_scorers.to_csv("students_above_240.csv", index=False)

print("\nFiles saved successfully!")
```

```
... Integrated Student-Marks Data:
  Student_ID Student_Name Department Python DBMS Statistics Total_Marks
0      S101         Ravi      CSE      85      82         90        257
1      S102        Priya      IT       78      80         75        233
2      S103         Arun      CSE      90      88         92        270
3      S104        Sneha      ECE      70      75         80        225
4      S105        Karthik     IT      88      84         86        258

Students with Total Marks greater than 240:
  Student_ID Student_Name Department Python DBMS Statistics Total_Marks
0      S101         Ravi      CSE      85      82         90        257
2      S103         Arun      CSE      90      88         92        270
4      S105        Karthik     IT      88      84         86        258

Files saved successfully!
```

4. Product-Sales Integration

You are given:

products.csv — Product_ID, Product_Name, Category, Unit_Price

sales.csv — Sale_ID, Product_ID, Quantity, Sale_Date

Task: Integrate the two datasets using Product_ID. Calculate the total sales amount for each transaction using:

$\text{Total_Sales} = \text{Quantity} \times \text{Unit_Price}$

Display the integrated dataset.

CODE:

```
import pandas as pd
```

```
# Read the datasets
```

```
products = pd.read_csv("products.csv")
```

```
sales = pd.read_csv("sales.csv")
```

```
# Merge the datasets using Product_ID
```

```
integrated_data = pd.merge(
```

```
    sales,
```

```
    products,
```

```

on="Product_ID",
how="inner"
)

# Calculate Total Sales
integrated_data["Total_Sales"] = (
    integrated_data["Quantity"] * integrated_data["Unit_Price"]
)

# Display the integrated dataset
print("Integrated Product-Sales Data:")
print(integrated_data)

# Save the integrated dataset
integrated_data.to_csv("product_sales_integrated.csv", index=False)

print("\nIntegrated dataset saved successfully!")

```

OUTPUT:

```

integrated_data["Total_Sales"] = \
    integrated_data["Quantity"] * integrated_data["Unit_Price"]
)

# Display the integrated dataset
print("Integrated Product-Sales Data:")
print(integrated_data)

# Save the integrated dataset
integrated_data.to_csv("product_sales_integrated.csv", index=False)

print("\nIntegrated dataset saved successfully!")

```

```

... Integrated Product-Sales Data:
  Sale_ID Product_ID Quantity  Sale_Date Product_Name  Category \
0   S001      P101         2  2026-08-01      Laptop  Electronics
1   S002      P102         3  2026-08-02        Mouse  Accessories
2   S003      P103         5  2026-08-03    Keyboard  Accessories
3   S004      P104         2  2026-08-04      Monitor  Electronics
4   S005      P105         1  2026-08-05      Printer    Office
5   S006      P101         1  2026-08-06      Laptop  Electronics

  Unit_Price  Total_Sales
0      50000      100000
1         800         2400
2        1500         7500
3       12000      24000
4         9000         9000
5       50000      50000

Integrated dataset saved successfully!

```


5. Hospital Patient–Visit Integration

You are given:

patients.csv — Patient_ID, Patient_Name, Age, Gender.

visits.csv — Visit_ID, Patient_ID, Doctor, Visit_Date, Fee

Task: Integrate the patient and visit datasets using Patient_ID. Display patient name, age, doctor, visit date, and fee. Calculate the total consultation fee paid by each patient.

CODE:

```
import pandas as pd

# Read the datasets
patients = pd.read_csv("patients.csv")
visits = pd.read_csv("visits.csv")

# Merge the datasets using Patient_ID
integrated_data = pd.merge(
    patients,
    visits,
    on="Patient_ID",
    how="inner"
)

# Display required columns
result = integrated_data[
    ["Patient_Name", "Age", "Doctor", "Visit_Date", "Fee"]
]

print("Integrated Patient-Visit Data:")
print(result)

# Calculate total consultation fee paid by each patient
total_fee = (
    integrated_data
    .groupby(["Patient_ID", "Patient_Name"])["Fee"]
    .sum()
    .reset_index()
)

total_fee.rename(
    columns={"Fee": "Total_Consultation_Fee"},
    inplace=True
)

print("\nTotal Consultation Fee Paid by Each Patient:")
print(total_fee)

# Save the integrated dataset
result.to_csv("patient_visit_integrated.csv", index=False)

# Save total fee report
total_fee.to_csv("total_consultation_fee.csv", index=False)
```

```
print("\nFiles saved successfully!")
```

OUTPUT:

```
▶ print("\nTotal Consultation Fee Paid by Each Patient:")
print(total_fee)

# Save the integrated dataset
result.to_csv("patient_visit_integrated.csv", index=False)

# Save total fee report
total_fee.to_csv("total_consultation_fee.csv", index=False)

print("\nFiles saved successfully!")
```

```
... Integrated Patient-Visit Data:
  Patient_Name  Age  Doctor  Visit_Date  Fee
0         Ravi   25  Dr. Kumar  2026-08-01  800
1         Ravi   25  Dr. Kumar  2026-08-10  600
2        Priya   32  Dr. Anitha  2026-08-02 1000
3        Priya   32  Dr. Anitha  2026-08-12  750
4         Arun   45   Dr. Raj  2026-08-03 1200
5        Sneha   28  Dr. Meena  2026-08-04  900
6       Karthik   51  Dr. Suresh  2026-08-05 1500
```

```
Total Consultation Fee Paid by Each Patient:
  Patient_ID Patient_Name  Total_Consultation_Fee
0        P101         Ravi                    1400
1        P102         Priya                    1750
2        P103          Arun                    1200
3        P104          Sneha                     900
4        P105       Karthik                    1500
```

```
Files saved successfully!
```